Assignment 5

Predict crop yield (tonnes/hectare) based on rainfall and fertilizer usage with different models like linear regression, ridge regression and random forest regressor on the crop recommendation dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score
```

Loading Dataset

```
rain = pd.read_csv("/content/rainfall in india 1901-2015.csv")
fert = pd.read_csv("/content/Fertilizer Prediction.csv")

rain.head()
```

	SUBDIVISION	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct De
0	ANDAMAN & NICOBAR ISLANDS	1901	49.2	87.1	29.2	2.3	528.8	517.5	365.1	481.1	332.6	388.5	558.2	33.6	3373.2	136.3	560.3	1696.3	980
1	ANDAMAN & NICOBAR ISLANDS	1902	0.0	159.8	12.2	0.0	446.1	537.1	228.9	753.7	666.2	197.2	359.0	160.5	3520.7	159.8	458.3	2185.9	716

Next steps: Generate code with rain | New interactive sheet

Data Preprocessing

```
fert.head()
```

	Temparature	Humidity	Moisture	Soil Type	Crop Type	Nitrogen	Potassium	Phosphorous	Fertilizer Name
0	26	52	38	Sandy	Maize	37	0	0	Urea
1	29	52	45	Loamy	Sugarcane	12	0	36	DAP
2	34	65	62	Black	Cotton	7	9	30	14-35-14
3	32	62	34	Red	Tobacco	22	0	20	28-28
4	28	54	46	Clayey	Paddy	35	0	0	Urea

Next steps: Generate code with fert | New interactive sheet

```
rain.isnull().sum()
```

	0
SUBDIVISION	0
YEAR	0
JAN	4
FEB	3
MAR	6
APR	4
MAY	3
JUN	5
JUL	7
AUG	4
SEP	6
OCT	7
NOV	11
DEC	10
ANNUAL	26
Jan-Feb	6
Mar-May	9
Jun-Sep	10
Oct-Dec	13

dtype: int64

```
rain.fillna(method="bfill",inplace=True)
```

```
/tmp/ipykernel_18238/1617055209.py:1: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future ver
  rain.fillna(method="bfill",inplace=True)
```

```
rain.isnull().sum()
```

	0
SUBDIVISION	0
YEAR	0
JAN	0
FEB	0
MAR	0
APR	0
MAY	0
JUN	0
JUL	0
AUG	0
SEP	0
OCT	0
NOV	0
DEC	0
ANNUAL	0
Jan-Feb	0
Mar-May	0
Jun-Sep	0
Oct-Dec	0

dtype: int64

```
fert.isnull().sum()
```

	0
Temparature	0
Humidity	0
Moisture	0
Soil Type	0
Crop Type	0
Nitrogen	0
Potassium	0
Phosphorous	0
Fertilizer Name	0

dtype: int64

Double-click (or enter) to edit

```
yearly_data = rain.groupby('YEAR')['ANNUAL'].mean().reset_index()

print(yearly_data.head())
```

```
   YEAR       ANNUAL
0  1901  1284.214286
1  1902  1337.302857
2  1903  1393.537143
3  1904  1236.771429
4  1905  1186.177143
```

```
X = yearly_data['YEAR'].values.reshape(-1, 1)
y = yearly_data['ANNUAL'].values
n = len(X)
```

Linear Regression

$$y = mx + c$$

This equation represents a straight line.

slope

$$m = \frac{n\sum xy - (\sum x)(\sum y)}{n\sum x^2 - (\sum x)^2}$$

This formula calculates the slope of the best-fit line using the least squares method.

$$c = \frac{\sum y - m\sum x}{n}$$

This formula calculates the intercept.

$$\hat{y} = mx + c$$

This equation is used to predict the output.

```
def linear_regression_formula(X, y):
    n = len(X)

    sum_x = sum(X)
    sum_y = sum(y)
    sum_xy = sum(X[i] * y[i] for i in range(n))
    sum_x2 = sum(X[i] ** 2 for i in range(n))

    m = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2)


    c = (sum_y - m * sum_x) / n

    return m, c


def predict(X, m, c):
    return [m * x + c for x in X]
```

Ridge Regression

$$\hat{y} = Xw$$

This equation represents the prediction using input matrix

$$J(w) = \sum(y - Xw)^2 + \lambda \sum w^2$$

This formula minimizes the squared error along with a penalty term.

$$w = (X^T X + \lambda I)^{-1} X^T y$$

This equation computes the optimal weights using matrix operations. Here:

$$\hat{y} = Xw$$

This equation is used to predict the output using the learned weights.

```
import numpy as np

class Ridge:
    def __init__(self, lam=1):
        self.lam = lam

    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y)

        m, n = X.shape

        # Add bias column (for intercept)
        X_b = np.c_[np.ones((m, 1)), X]

        I = np.eye(n + 1)
        I[0][0] = 0   # Do not regularize bias

        # Closed-form solution
        self.w = np.linalg.inv(X_b.T @ X_b + self.lam * I) @ X_b.T @ y
        return self

    def predict(self, X):
        X = np.array(X)
        m = len(X)

        X_b = np.c_[np.ones((m, 1)), X]
        return X_b @ self.w
```

Model Prediction

```
y_pred_ridge = Ridge().fit(X, y).predict(X)

print("\n--- Ridge Regression ---")
print("R2 Score:", r2_score(y, y_pred_ridge))
```

```
--- Ridge Regression ---
```

```
y_pred_linear = predict(X, *linear_regression_formula(X, y))
print("\n--- Linear Regression ---")
print("R2 Score:", r2_score(y, y_pred_linear))
```

```
--- Linear Regression ---
R2 Score: 0.0076292321386612905
```

Predicting Rainfall for 2030

```
ridge_model = Ridge().fit(X, y)
pred_2030_ridge = ridge_model.predict(year_2030)

print(f"Predicted Annual Rainfall for 2030 (Ridge Regression): {pred_2030_ridge[0]:.2f} mm")
```

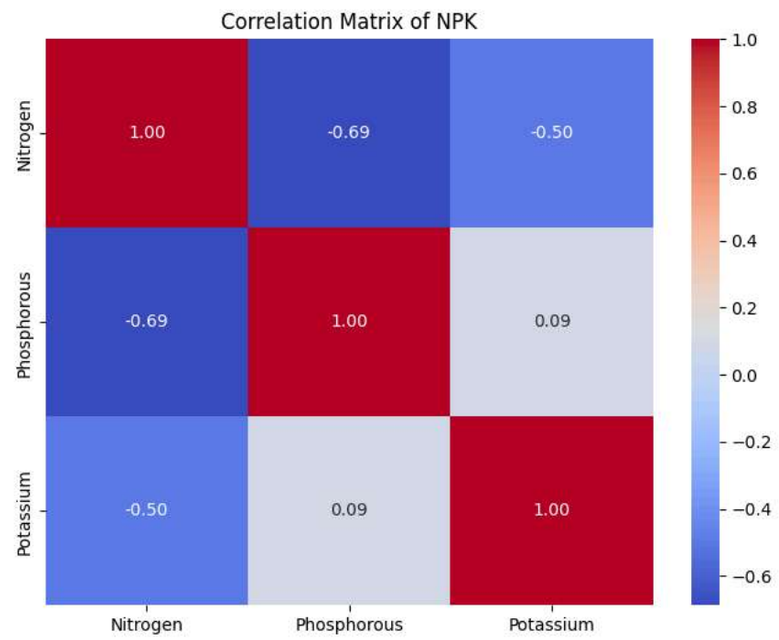
```
Predicted Annual Rainfall for 2030 (Ridge Regression): 1396.16 mm
```

Correlation Analysis

```
import seaborn as sns
import matplotlib.pyplot as plt

corr_matrix = fert[['Nitrogen','Phosphorous','Potassium']].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of NPK')
plt.show()
```




```
N = fert['Nitrogen'].values
P = fert['Phosphorous'].values
K = fert['Potassium'].values
```

```
print("\nCorrelation Matrix:\n", corr_matrix)
```

```
Correlation Matrix:
              Nitrogen  Phosphorous  Potassium
Nitrogen     1.000000    -0.686971  -0.500087
Phosphorous -0.686971     1.000000   0.089192
Potassium   -0.500087     0.089192   1.000000
```

Conclusion

Rainfall prediction via Linear/Ridge models showed low accuracy (R2 ~0.0076), projecting ~1396.16 mm for 2030. Fertilizer analysis revealed strong negative correlations: N with P (-0.69), and N with K (-0.50). Crop yield prediction remains pending, as a direct 'crop yield' column is still needed.

```
Start coding or generate with AI.
```